# WebGL 3D Star System Generator

Course Name: CSE 606, Computer Graphics
Student Name: Gathik Jindal
Student ID: IMT2023089

November 03, 2025

**Abstract**

This project implements a real-time 3D star system generator using the native WebGL 1.0 API. It features a central emissive star and multiple planets loaded from `.ply` models, which revolve on unique, concentric elliptical orbits. The application supports dual-camera modes: a "3D View" with quaternion-based trackball controls and a "Top View" for orthographic selection. Object picking is implemented using a pixel-perfect, off-screen framebuffer technique. Users can interactively add and delete planets, and control the simulation's animation (revolution, manual rotation, and scaling) via key bindings.

## 1 Features

This project implements all core and optional features described in the assignment.

- **3D Model Loading:** Loads multiple `.ply` models (sphere, icosphere, monkey, torus, etc.) on startup using a custom parser ('PLYLoader.js').
- **Dual-Camera System:** Toggle between a "3D View" and a "Top View".
- **3D Trackball Camera:** The 3D view features a quaternion-based arcball camera for smooth, arbitrary rotations around the scene origin.
- **Orientation Gizmo:** A 3D axis gizmo, rendered in a separate canvas, mirrors the main camera's orientation.
- **Elliptical Orbits:** Planets revolve on unique, concentric elliptical orbits, each rendered as a colored line on the XZ-plane.
- **Object Picking:** In "Top View," users can click to select any planet. This is implemented using a high-precision, off-screen framebuffer (FBO) with color-ID encoding.
- **Object Highlighting:** The selected planet is instantly highlighted with a distinct white color.
- **Interactive Planet Management:**
  - **Add Planet:** Clicking a model in the right-hand sidebar adds it as a new planet with a unique, non-colliding orbit.
  - **Delete Planet:** The selected planet can be deleted, respecting the 3-planet minimum rule.
- **Animation & Transformation Controls:**
  - **Revolution Toggle:** Automatic planet revolution can be toggled on/off ("stationary") with a keybind.
  - **Speed Control:** The revolution speed of the selected planet can be increased or decreased.
  - **Manual Rotation:** When stationary, the selected planet can be manually rotated on its x, y, or z-axis using quaternions.
  - **Scaling:** When stationary, the selected planet can be scaled up or down. It resumes its original size when revolution is re-enabled.

# 2 Controls

Table 1: Interaction Controls

| Key / Input | Action |
|---|---|
| Click (Sidebar) | Add a new planet |
| Click (Top View) | Select a planet |
| Mouse Drag (3D View) | Rotate the 3D camera |
| Mouse Wheel (3D View) | Zoom the 3D camera |
| ‘Spacebar‘ | Toggle planet revolution (stationary) |
| ‘t‘ | Toggle manual rotation mode |
| ‘x‘/‘X‘, ‘y‘/‘Y‘, ‘z‘/‘Z‘ | Manually rotate selected planet (in manual mode) |
| ‘PageUp‘ / ‘PageDown‘ | Scale selected planet (when stationary) |
| ‘[‘ / ‘]‘ | Decrease / Increase revolution speed |
| ‘Delete‘ / ‘Backspace‘ | Delete selected planet |

# 3 Methodology

The application is built with modern JavaScript (ES6 Modules) and raw WebGL 1.0, centered around a main ‘StarSystem‘ class.

## 3.1 Rendering Pipeline

The system uses three separate WebGL programs:

1. **Main Shader:** A Phong-style shader (‘Shaders.js‘) for rendering all 3D objects. It supports diffuse lighting from the star (at origin) and a uniform ‘u_isEmissive‘ to make the star and gizmo axes glow.
2. **Orbit Shader:** A minimal shader (‘OrbitShaders.js‘) for drawing the flat, colored lines of the elliptical orbits.
3. **Gizmo Renderer:** A separate ‘gizmoRenderer.js‘ class manages its own WebGL context and objects, syncing its camera’s view matrix to the main scene’s camera rotation.

All transformations are calculated on the CPU using ‘gl-matrix‘ and sent to the GPU. The model matrix for planets is rebuilt from scratch each frame as ‘M = Translation * Rotation * Scale‘.

## 3.2 Scene Management and Animation

The ‘StarSystem.js‘ class manages the state of all planets. Each planet is an object containing a ‘GameObject‘ instance and its animation properties (e.g., ‘orbitA‘, ‘orbitB‘, ‘orbitAngle‘, ‘rotationQuat‘, ‘tempScale‘).

The ‘_update‘ loop calculates all transformations. Planet revolution uses the formula ‘x = a * cos(angle)‘ and ‘z = b * sin(angle)‘ to get the translation. This is combined with the ‘rotationQuat‘ (for spin) and ‘tempScale‘ (for size) to generate the final model matrix.

## 3.3 Camera System

The application supports two distinct camera modes:

- **Top View:** An orthographic camera looking straight down the Y-axis. The ”up” vector is set to ‘(0, 0, -1)‘ to orient the view correctly.
- **3D View:** A perspective camera using a quaternion-based arcball (or ”virtual trackball”). Mouse movement updates the ‘cameraOrientation‘ quaternion, which is then used to transform the camera’s position and ”up” vector, providing smooth, gimbal-lock-free rotation.

### 3.4 Interactivity and Picking

User input is modularized into ‘InputHandler.js‘. The most complex interaction is object picking, which is handled by the ‘ObjectPicker.js‘ class.

1. When the user clicks in ”Top View,” the ‘ObjectPicker‘ re-renders the scene to a hidden, off-screen framebuffer (FBO).
2. In this hidden render, each pickable planet is drawn with a unique, flat color (e.g., ‘(1/255, 0, 0)‘, ‘(2/255, 0, 0)‘, etc.) that acts as its ID.
3. ‘gl.readPixels‘ is used to read the color of the single pixel at the mouse location.
4. The red value of the pixel ‘(e.g., 2)‘ directly corresponds to the ID of the planet that was clicked.
5. ‘StarSystem.setSelected()‘ is then called with the correct planet.

## 4 File Structure

The project is organized into several JavaScript modules to separate concerns. The six most important files are:

- **index.js**: The main entry point for the application. It is responsible for initializing the various WebGL contexts (main, gizmo, orbits), loading all 3D models, instantiating the main renderer classes, and starting the render loop.
- **StarSystem.js**: The core class of the project. It manages the entire scene state, including the list of planets, the star, and the camera. It contains all the logic for animation (‘_update‘), rendering (‘_draw‘), and object manipulation (e.g., ‘addModel‘, ‘deleteSelected‘, ‘rotateSelected‘).
- **GameObject.js**: A class representing a single renderable 3D object in the scene (like a planet, the star, or an axis part). It is responsible for creating and holding its own WebGL buffers (position, normal, index) and contains the ‘draw()‘ method to render itself.
- **InputHandler.js**: A modular utility that contains all event listeners for the application. It handles all keyboard input for transformations (scaling, rotation), animation toggles, and mouse input for camera control and object picking, keeping the main ‘index.js‘ file clean.
- **ObjectPicker.js**: Implements the advanced object picking functionality. This class manages a hidden, off-screen framebuffer (FBO). It can render the scene to this FBO using unique color IDs for each planet, and then read the pixel at the mouse location to identify which object was clicked.
- **Shaders.js**: Contains the GLSL (OpenGL Shading Language) source code for the main 3D objects. This includes the vertex shader, which positions vertices, and the fragment shader, which calculates lighting (diffuse + ambient) and color for each pixel, including the ”emissive” property for the star.

## 5 Run Locally

This project uses ES6 modules, which requires it to be run from a local web server to function correctly.

1. **Clone the repository or download the source files.**
2. **Navigate to the project directory.**
3. **Start a local web server.** A simple way is to use Python’s built-in HTTP server.
   - If you have Python 3:
     ```
     python -m http.server
     ```
   - If you have Python 2:
     ```
     python -m SimpleHTTPServer
     ```

   Alternatively, you can use a development tool like the Live Server extension for VS Code.
4. **Open your web browser** and navigate to the local server’s address (e.g., `http://localhost:8000`).

# 6 Dependencies

This project relies on one external library included via CDN:

- *gl-matrix*: A JavaScript library for high-performance vector and matrix math, used for all quaternion, vector, and matrix operations.

# 7 Questions To Be Answered

## 7.1 To what extent were you able to reuse code from Assignment 1?

Very little code was reused. Assignment 1 focused on 2D rendering, so while the concepts of buffers and shaders are similar, the implementation for 3D is substantially different. The main reusable components were the basic WebGL context initialization and the shader compilation functions, which have now been modularized into 'ShaderUtil.js'.

## 7.2 What were the primary changes in the use of WebGL in moving from 2D to 3D?

- **Matrices:** We moved from 2D (x, y) coordinates to 3D (x, y, z) coordinates. This required three separate matrices (Model, View, Projection) instead of a single 2D transformation matrix.
- **Depth:** We enabled the 'DEPTH_TEST' ('gl.enable(gl.DEPTH_TEST)') and must clear the 'DEPTH_BUFFER_BIT' each frame. This is crucial for rendering 3D objects in the correct order.
- **Camera:** A 3D camera system was implemented. The "View" matrix represents the camera, managed with 'mat4.lookAt' and updated with quaternions for smooth 3D rotation.
- **Lighting:** In 2D, color is often flat. In 3D, we implemented a basic Phong-style lighting model in the fragment shader ('Shaders.js'). This requires vertex normals ('a_Normal') and a light position ('u_LightPosition') to calculate diffuse light.
- **Models:** We are no longer drawing simple 2D shapes but loading complex 3D meshes from '.ply' files.

## 7.3 How were the translate, scale and rotate matrices arranged? Can your implementation allow rotations and scaling during the movement?

- **Arrangement:** The transformations are applied in the 'StarSystem.js' '_update' loop. The final Model matrix ('M') is created by combining separate matrices for Translation, Rotation, and Scale. The order of multiplication is: $M = \text{Translation} \times \text{Rotation} \times \text{Scale}$ This ensures that the object is scaled in its local space, then rotated around its own center, and finally moved to its correct position in the orbit.
- **Allow during movement?**
  - **Rotation:** Yes, the architecture allows it. Planets could be rotated while moving, but this was disabled by default to meet the assignment requirement that "only axis-rotation or revolution can happen at a time."
  - **Scaling:** No, the implementation explicitly prevents this. The code checks 'if (!this.isRevolving)' before applying the 'tempScale'. This fulfills the requirement that the planet "resume its original size when it starts moving."

## 7.4 How did you ensure that they are no conflicts when adding/deleting a planet along with its orbit?

- **Adding:** Conflicts are avoided by finding the largest existing orbital radii. In 'StarSystem.js', the 'addModel' function iterates over all planets to find the 'maxA' and 'maxB' (x and z radii). It then creates the new planet's orbit by adding a random offset to these maximums, ensuring the new orbit is always larger than all existing ones and cannot overlap.

- **Deleting:** This is managed safely. When ‘deleteSelected‘ is called, the program finds the ‘planetProp‘ object. It then explicitly calls ‘this.orbitRenderer.removeOrbit(planetProp.orbitMesh)‘ to delete the WebGL buffer for the orbit line, and finally filters the planet from the ‘this.planets‘ array. A check also prevents deletion if there are 3 or fewer planets, as required.

# 8 Demo

Check out the file called ‘demo.mp4‘ in the repository.

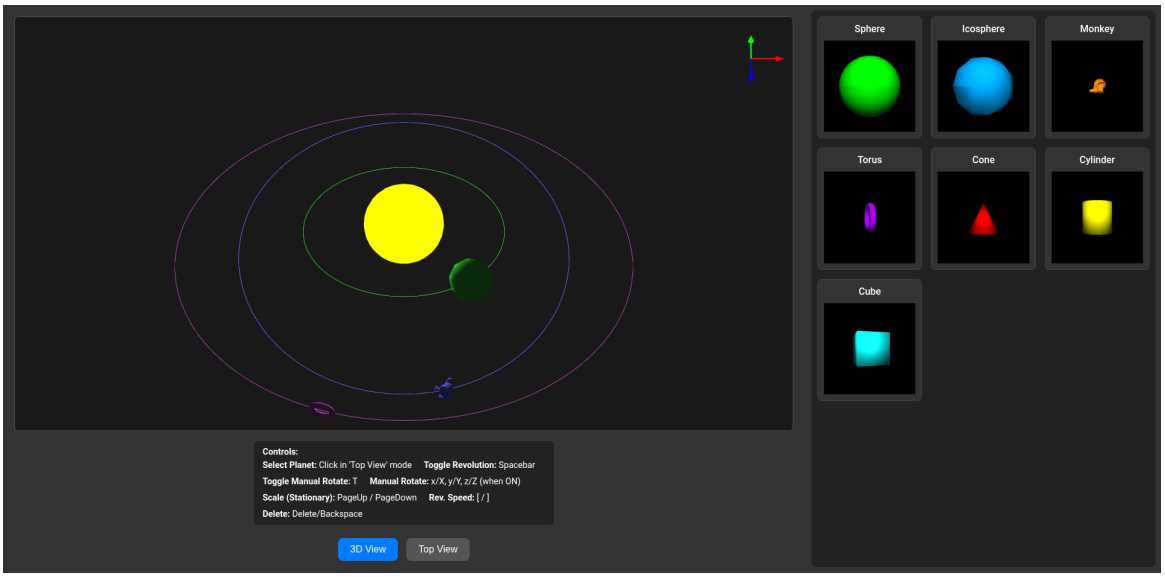


Figure 1: The 3D View, showing the central star, planets with elliptical orbits, and the orientation gizmo.

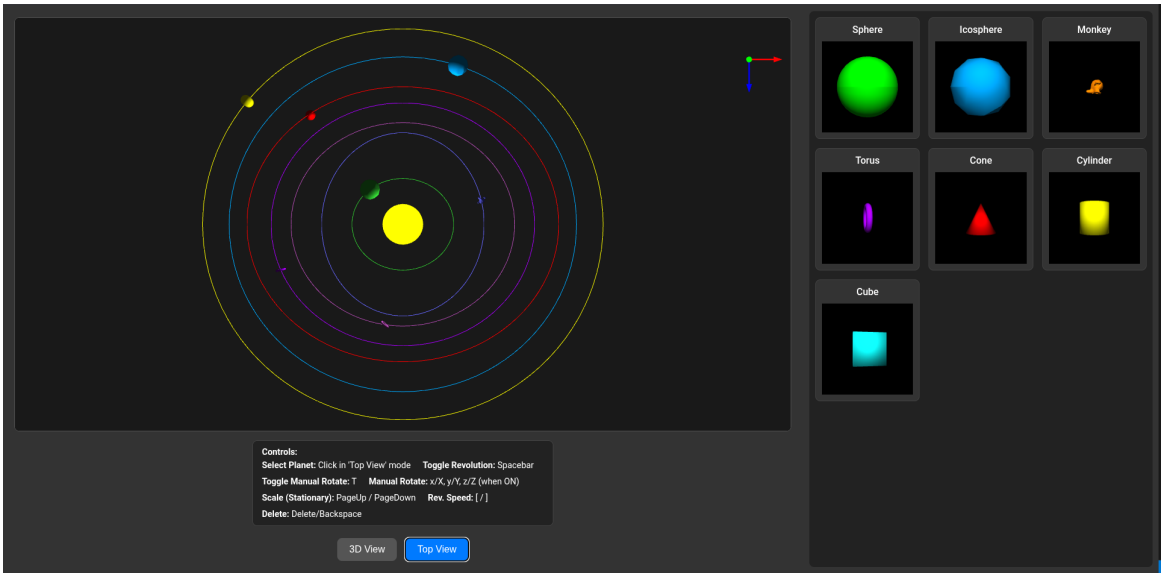


Figure 2: The Top View, used for object picking.

# 9 Acknowledgments

I would like to acknowledge the resources that were critical to the completion of this project. The Mozilla Developer Network (MDN) Web Docs and the ‘gl-matrix‘ documentation served as essential technical references for the WebGL API and matrix/quaternion mathematics and the use of an AI assistant was immeasurable.